DOCUMENTATION · SECURITY

Security questionnaire answers

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=QUESTIONNAIRE. This copy was made on 4 November 2026.

These are answers to the questions a vendor security questionnaire (SIG Lite, CAIQ, or a customer's own) usually asks, for **QueryAPIGate 0.40.0**. QueryAPIGate is a self-hosted, open source (Apache-2.0) gateway that turns saved SQL into governed REST, MCP, event and export endpoints.

How to read them:

- The answers describe **the software**: what it does, what it stores, and where its controls stop. QueryAPIGate runs inside your environment, so the answers about **your deployment** (which features are on, who operates it, how the host and network are secured) are yours to give. In this document, **"Deployer"** means you, the customer running QueryAPIGate.
- Items marked **Deployer:** name a control that is your responsibility, or that depends on how you configure it.
- Each answer cites its source. Files in this pack sit side by side (`THREAT_MODEL.md`, `DATA_PROTECTION.md`, `SECURITY.md`, `DEPLOYMENT.md`, `ARCHITECTURE.md`). Other sources are linked in the project's repository.
- Where a source didn't support a claim, the claim was left out. To check the answers against your own running server, see guide 57.

---

1. Product and deployment model

Is the product delivered as a hosted service (SaaS) or as software the customer runs?

Answer: It is software the customer runs. It is not a service the project hosts. It is published as a container image (`ghcr.io/anantharajuc/queryapigate`, plus an `-h2` variant), a Helm chart and a Python package, and it runs on the Deployer's own infrastructure.

Source: DEPLOYMENT.md#1-the-image, DEPLOYMENT.md#kubernetes-helm

---

Does the product send any data to the vendor, such as telemetry, usage statistics or licence checks?

Answer: No. Every place data leaves the process is listed in the data-flow inventory, and a test fails if the code adds a path the inventory doesn't list. Every destination on that list is infrastructure the Deployer configures: their log platform, trace collector, Prometheus, SIEM, export buckets, secret providers, and the AI agents and clients they authorise. The project's trust page says this as: no telemetry, no licence server, nothing sent to the project. One exception happens in the browser, not the server: the `/docs` page loads one pinned Swagger UI release from a CDN, with Subresource Integrity hashes.

Source: DATA_PROTECTION.md#where-data-is-sent, THREAT_MODEL.md#transport, ADR 0010

Where is customer data processed and stored?

Answer: Only where the Deployer runs it. Query results pass through the process to the caller and are not kept, except in two features the Deployer turns on: the response cache and exports. Configuration, run history and the audit log are kept in a "store", which is either a SQLite file (`queryapigate.db`) on a volume or a PostgreSQL database that the Deployer provides.

Source: DATA_PROTECTION.md#what-passes-through-and-isnt-kept, DATA_PROTECTION.md#the-store

---

2. Identity and authentication

What authentication methods does the API support?

Answer: The API supports four kinds of credential:

- the shared admin key (`QUERYAPIGATE_API_KEY`);
- named administrators' admin tokens (`qagadm_...`);
- scoped API keys (`sk_...`), sent in the `X-API-Key` header;
- for applications whose users sign in, a JWT in `Authorization: Bearer` , verified against a shared secret or the identity provider's JWKS.

The MCP server accepts the same keys, or OAuth 2.1 access tokens.

Source: THREAT_MODEL.md#authentication-one-shared-secret-or-many-scoped-ones

---

Is authentication required by default?

Answer: No. If no API key is set, no scoped keys or administrators exist and JWT is off, every caller is treated as an unrestricted administrator. This "open access" mode is deliberate, for local trials, and is documented as unsafe for anything reachable beyond a machine you control. The `open_server` alert (critical) reports this state. Authentication is required once any scoped key or administrator exists, or when JWT is on.

Deployer: set `QUERYAPIGATE_API_KEY` before the server can be reached.

Source: THREAT_MODEL.md#open-access-mode-a-deliberate-default-worth-understanding, API.md, Alerts

---

How are API keys and tokens generated and stored?

Answer: The server generates scoped keys and admin tokens from the operating system's random source and shows each one once. Only its SHA-256 hash is stored, so a lost key can't be recovered; it can only be revoked and replaced. The shared admin key lives only in the process environment and is compared in constant time (`hmac.compare_digest`).

Source: THREAT_MODEL.md#authentication-one-shared-secret-or-many-scoped-ones, DATA_PROTECTION.md#cryptography

---

Can each administrator have an individual identity, rather than a shared account?

Answer: Yes. Named administrators each have a role and their own admin tokens. An administrator can hold several tokens and revoke each one separately, and deactivating an administrator stops all of their tokens at once. Once an active owner exists, the shared key becomes a break-glass key: each use is logged, audited (`break_glass_used`) and raises an alert, and owners can remove it.

Source: THREAT_MODEL.md#administration-named-administrators-and-roles, API.md, Administrators and admin roles

---

Does the product support single sign-on (SSO) for administrators?

Answer: Yes. Console sign-in supports one identity provider at a time, through OpenID Connect. SAML is not supported. The server runs the authorization code flow with PKCE (S256) as a confidential client, so the browser never holds a token, and each administrator's role comes from their groups at the provider. This feature has been stable since 0.30, after a security review.

Source: THREAT_MODEL.md#console-sign-in-through-sso-adr-0007, API.md, Console sign-in through SSO

Does the product support multi-factor authentication (MFA)?

Answer: Not directly. QueryAPIGate has no MFA of its own. When administrators sign in through SSO, the identity provider's MFA and policies apply. `QUERYAPIGATE_SSO_ONLY` removes personal tokens from people who sign in through SSO, so they can't bypass the provider's MFA. Admin tokens and API keys are bearer secrets with no second factor.

Deployer: enforce MFA at your identity provider, and set `QUERYAPIGATE_SSO_ONLY`.

Source: THREAT_MODEL.md#console-sign-in-through-sso-adr-0007, API.md, Console sign-in through SSO

How are administrator Console sessions protected?

Answer: The session cookie is `__Host-`, `HttpOnly`, `Secure` and `SameSite=Strict`, and is stored only as a hash. It authenticates a request only together with the session's CSRF token. A sign-in callback is used once, within ten minutes, and only in the browser that started it. Sessions last `QUERYAPIGATE_SSO_SESSION_HOURS` (8 by default, at most 168).

Source: THREAT_MODEL.md#console-sign-in-through-sso-adr-0007, API.md, Console sign-in through SSO

When a person is removed at the identity provider, how quickly does their access end?

Answer: A Console session that is already open lasts until it expires (`QUERYAPIGATE_SSO_SESSION_HOURS`). Switching the administrator off in QueryAPIGate ends their sessions at once. JWTs and MCP OAuth tokens can't be revoked before they expire; disabling or narrowing the role in QueryAPIGate takes effect on the next request.

Deployer: keep access-token lifetimes short.

Source: THREAT_MODEL.md#console-sign-in-through-sso-adr-0007, THREAT_MODEL.md#oauth-for-ai-clients-adr-0006

How are tokens from an external identity provider validated (JWT/JWKS)?

Answer: The checks are:

- the signature;
- `exp`, which is required;
- `iss` and `aud`, both mandatory when a JWKS URL is used;
- the algorithm, which must be one the configuration fixes. HMAC and public-key families can't be mixed, and `none` is never accepted.

Settings that would make checking unsafe stop the server at startup, for example a JWKS URL over plain `http://` or a short secret. A parameter declared `from_claim` is bound from the verified token, never from the request.

Source: THREAT_MODEL.md#authentication-one-shared-secret-or-many-scoped-ones, API.md, Signed-in users (JWT)

How do AI agents (MCP clients) authenticate?

Answer: Agents send an API key in a request header, or use OAuth 2.1. With OAuth, the MCP server acts as a resource server and checks:

- `aud`, which must be the MCP server's own URL;
- `iss`;
- the signature, using the provider's own key algorithm (asymmetric only);
- `exp`;
- the configured scope.

A token issued for another application, including this server's own REST API, is refused. A caller's token is never passed on to databases.

Source: THREAT_MODEL.md#oauth-for-ai-clients-adr-0006, THREAT_MODEL.md#mcp-server

---

Do credentials expire?

Answer: Scoped API keys can carry an `expires_at` date, checked live on every request, and the `key_expiring` and `key_expired` alerts report them. Admin tokens expire after 90 days by default (`--expires YYYY-MM-DD|never`). Expiry is set to the day, not the hour or minute. JWT and OAuth tokens expire as their provider issues them.

Source: API.md, Key expiry, API.md, Administrators and admin roles, THREAT_MODEL.md#administration-named-administrators-and-roles

---

Can credentials be rotated without downtime?

Answer: Yes. Reissuing a scoped key's secret (`POST /api/v1/api-keys/{name}/rotate`) shows the new secret once. The old secret keeps working for a grace period the administrator chooses, at most a week, or 0 for a leaked secret. The administrator can see when the old secret was last used. The audit key and the encryption key for stored

passwords can also be rotated: list the new key first and the old one after it.

Source: THREAT_MODEL.md#authentication-one-shared-secret-or-many-scoped-ones, THREAT_MODEL.md#secrets-at-rest, guide 50

---

3. Authorization and least privilege

What access-control model does the product use?

Answer: Administrators have one of four fixed roles, each holding a fixed set of capabilities:

- **owner:** everything;
- **admin:** everything except administrators;
- **developer:** saved queries and collections;
- **auditor:** read-only, with no SQL.

Data callers (scoped keys, signed-in users and agents) get grants instead. A test fails if a Management API operation has no capability assigned, so a new route can't ship unguarded.

Source: THREAT_MODEL.md#administration-named-administrators-and-roles

---

What can a scoped API key be restricted to?

Answer: A scoped key can be limited to:

- connections (ad-hoc SQL targets);
- collections and named saved queries;
- whether it may write, and which write verbs;
- `allowed_tables`;
- `allowed_ips`;
- a rate limit, concurrency, time and row limits;
- usage quotas;
- an expiry date;
- column masking.

`connections`, `collections` and `queries` add up to what the key can reach. The other fields only ever narrow that reach. A `queries` grant alone gives an external client curated access to named saved queries, with no ad-hoc SQL at all.

Source: THREAT_MODEL.md#authorization-what-a-scoped-key-can-actually-do

---

Is access read-only by default?

Answer: Yes. Writes need `QUERYAPIGATE_ALLOW_WRITES` on the server, and also `allow_writes` on the key. A key can never grant itself more than the server allows. The read-only guard rejects write statements, including writes hidden in a writable CTE, in `EXPLAIN ANALYZE` or in `SELECT ... INTO`. PostgreSQL, MySQL, ClickHouse and SQLite

connections also run read-only queries in read-only database sessions.

Source: THREAT_MODEL.md#authorization-what-a-scoped-key-can-actually-do, THREAT_MODEL.md#query-execution-what-actually-stops-a-bad-statement

---

How is access to specific tables restricted?

Answer: With `allowed_tables`. It is enforced with a real SQL parser (`sqlglot`), not a regex. A key restricted to tables is refused outright on H2 and JDBC connections, rather than checked loosely. On MongoDB the list is the collections a key may `find()` in; this was fixed in 0.34.1 (see section 9).

Source: THREAT_MODEL.md#authorization-what-a-scoped-key-can-actually-do, CHANGELOG 0.34.1

Can access be restricted by source IP address?

Answer: Yes. A key's `allowed_ips` lists IPv4 or IPv6 addresses and CIDR ranges. The check fails closed if the client address can't be resolved. The shared admin key is never restricted this way. Behind a proxy, the MCP server sees the proxy's address for every caller, so `allowed_ips` can't tell MCP callers apart there.

Deployer: set `QUERYAPIGATE_TRUST_PROXY` to the real number of proxy hops. A wrong value lets a client forge its address.

Source: THREAT_MODEL.md#authorization-what-a-scoped-key-can-actually-do, THREAT_MODEL.md#rate-limiting-and-abuse-prevention, THREAT_MODEL.md#mcp-server

---

How are roles applied to keys? Do changes to a role apply retroactively?

Answer: It depends on the caller. For API keys, roles are templates: a key created from a role copies its grants once, and editing the role later doesn't change existing keys. Signed-in JWT users use their role live, so a change to the role applies on their next request.

Source: THREAT_MODEL.md#authorization-what-a-scoped-key-can-actually-do, API.md, Signed-in users (JWT)

Can an AI agent change data?

Answer: Only through saved queries an author has marked `agent_writes`. Each must be a single `INSERT`, `UPDATE` or `DELETE`, never DDL, and the caller's grant must allow the write, with `QUERYAPIGATE_ALLOW_WRITES` on. A named person approves each write before it runs, and the credential that asked can never approve its own request. Each write runs in a transaction that is rolled back if it changes more than `max_affected_rows`. Insert-only queries skip the approval. `execute_sql` over MCP never writes.

Source: THREAT_MODEL.md#agents-that-write-adr-0008, THREAT_MODEL.md#mcp-server

---

How do you protect against one caller exhausting shared resources?

Answer: Several limits apply:

- per-address and per-key rate limits;
- per-key or per-role `max_concurrent`, `query_timeout` and `max_rows`, which only ever narrow the server's own limits;
- a server-wide query time limit (`QUERYAPIGATE_QUERY_TIMEOUT`, 30 seconds by default) and `QUERYAPIGATE_MAX_PAGE_SIZE`.

Usage quotas meter calls and rows per day or month, but they aren't a hard security boundary.

Deployer: set `QUERYAPIGATE_RATE_LIMIT` (off by default) on anything reachable beyond a trusted network; it also throttles API key guessing.

Source: THREAT_MODEL.md#rate-limiting-and-abuse-prevention, SECURITY.md#hardening-checklist-for-deployments

---

Does the product enforce least privilege on the database side?

Answer: No. QueryAPIGate connects with the database account each connection is given. Its own read-only guard is defence in depth, not a substitute for database grants.

Deployer: connect with a database account that has only the privileges the API needs.

Source: SECURITY.md#hardening-checklist-for-deployments, THREAT_MODEL.md#non-goals-what-this-does-not-defend-against

---

4. Data protection and privacy

What customer data does the product store?

Answer: The store holds configuration (connections, saved queries and their versions, keys, roles and administrators), run history, usage counters, the event log, the audit log, and an agent's write requests. Query results are not stored, except by the response cache and exports when the Deployer turns them on. Parameter values are not stored either: run history records parameter names, never their values.

Source: DATA_PROTECTION.md#the-store, DATA_PROTECTION.md#what-passes-through-and-isnt-kept

---

What personal data does the product process?

Answer: The product processes:

- callers' names, which for a signed-in user is `jwt:<subject>`, often an email address;
- administrators' names and email addresses;
- IP addresses;
- an AI agent's write arguments;
- whatever the connected databases hold, which passes through.

With no SSO and no JWTs, no person is named, only API keys.

Source: DATA_PROTECTION.md#personal-data-it-handles, guide 57

Is personal data written to logs?

Answer: Requests are logged with their path, method, status, time, request id, client address and caller name; the query string is left out unless `QUERYAPIGATE_ACCESS_LOG_QUERY=1` is set. Some things can still reach the log: an ad-hoc statement's text, and a database's own error message, which can quote a value. Setting `QUERYAPIGATE_HISTORY_ADHOC_SQL=hash` reduces ad-hoc statements to their SHA-256.

Deployer: treat the log platform as holding personal data.

Source: DATA_PROTECTION.md#where-data-is-sent, THREAT_MODEL.md#what-is-kept-and-the-cryptography-it-rests-on-adr-0016

Can sensitive columns be hidden from specific consumers?

Answer: Yes. A `mask` grant on a key, role or export can hash, mask, null or drop named source columns. Result columns are traced to their sources through the SQL, not matched by name, and a query that can't be traced is refused before it runs. Masking hides values, not answers about them: a caller who writes SQL can still filter or group on a masked column. Callers with masking rules are refused on H2, JDBC and MongoDB connections.

Source: THREAT_MODEL.md#column-masking-which-values-leave, API.md, Column masking

---

How long is each kind of data retained?

Answer: Retention, by kind of data:

DATA	KEPT FOR
Run history	Bounded per query version, for ad-hoc runs, by days, or by sampling (`QUERYAPIGATE_HISTORY_*`)
Event log	24 hours by default
Usage counters	400 days
Agent write requests	30 days
Audit log in the store	The newest `QUERYAPIGATE_AUDIT_LOG_LIMIT` entries (500 by default)
SSO sign-ins in progress	10 minutes
Configuration	Until deleted

Deployer: the retention of logs, traces and SIEM copies is set by your own platforms.

Source: DATA_PROTECTION.md#the-store, DATA_PROTECTION.md#what-a-deployer-controls

---

Can an individual's data be erased on request (right to erasure)?

Answer: Yes. An owner can run `queryapigate history forget <caller>` or `DELETE /api/v1/history?caller=...`. This deletes the caller's runs, events and usage counts in the store and in Redis, and records the erasure in the audit log by the caller's SHA-256, not their name. A `--dry-run` shows the counts first. The command does not erase the audit log's entries about administrators, write requests (kept 30 days), or the Deployer's logs, traces and SIEM copies.

Source: DATA_PROTECTION.md#a-person-asks-for-their-data-to-be-erased

---

Who can read stored personal data?

Answer: Access is by administrator capability:

- run history and the audit log need `observe`;
- keys need `access.read`;
- agents' write arguments need `writes.approve`;
- erasure needs `history.forget`, which only owners have.

Callers see only what their grants allow, after masking.

Source: DATA_PROTECTION.md#the-store, guide 57

Can data be delivered to third parties, such as partners' storage buckets?

Answer: Only through exports. Only owners and admins can define an export (`exports.write`). Each export writes only under its destination's prefix, after the export's own masking, and every change to an export is audited.

Deployer: scope the destination credentials to the prefix.

Source: THREAT_MODEL.md#query-execution-what-actually-stops-a-bad-statement, DATA_PROTECTION.md#where-data-is-sent

---

5. Cryptography

Is data encrypted at rest?

Answer: Partly. Connection and export-destination passwords are Fernet-encrypted (AES-128-CBC with HMAC-SHA256) when `QUERYAPIGATE_SECRET_KEY` is set. This is opt-in, not the default. API keys, admin tokens and session ids are stored only as SHA-256 hashes. The rest of the store (configuration and run history) is not encrypted by QueryAPIGate.

Deployer: put the store on an encrypted volume or database.

Source: THREAT_MODEL.md#secrets-at-rest, DATA_PROTECTION.md#cryptography, guide 57

Can database credentials be kept out of the product's store entirely?

Answer: Yes. A password can be a reference: `${VAR}`, `${file:...}` or `${vault:...}`, or short-lived Vault database credentials. References to AWS Secrets Manager and RDS IAM are experimental. Resolved values are held only in memory and never written to the store, a log, a response, a trace or the audit log. Nothing is reachable through a reference until the Deployer allow-lists it.

Source: THREAT_MODEL.md#secrets-from-secrets-managers-adr-0015, DEPLOYMENT.md#3-secrets-env

---

How are encryption keys managed?

Answer: `QUERYAPIGATE_SECRET_KEY` and `QUERYAPIGATE_AUDIT_KEY` are supplied by the Deployer and must be kept outside the store and out of version control. Without the encryption key, encrypted passwords can't be recovered. Both keys can be rotated: `queryapigate rekey` re-encrypts every stored password in one all-or-nothing transaction.

Deployer: keep the keys in your secrets manager.

Source: THREAT_MODEL.md#secrets-at-rest, API.md, A tamper-evident chain

---

Is data encrypted in transit?

Answer: Inbound traffic is not encrypted by QueryAPIGate itself: it does not terminate TLS, which is the reverse proxy's job. Outbound connections to Vault, the SIEM, the OTLP collector and Redis use TLS through Python's `ssl`. PostgreSQL and MySQL connections' TLS settings reach their drivers, and `aws-rds-iam` is refused without TLS.

Deployer: terminate TLS in front of the server. It is required once a key exists.

Source: THREAT_MODEL.md#transport, DATA_PROTECTION.md#cryptography, THREAT_MODEL.md#secrets-from-secrets-managers-adr-0015

Is the cryptography FIPS 140 validated?

Answer: QueryAPIGate uses only FIPS-approved algorithms (FIPS 140-3, SP 800-131A), and a test fails if the code gains one that isn't approved. Validation is a property of the cryptographic module, not the application: QueryAPIGate runs on a FIPS 140-validated module when the platform provides one. Two examples are RHEL 9 in FIPS mode and Ubuntu Pro FIPS, with Python and `cryptography` from the distribution. QueryAPIGate itself is not a validated module. A CI job runs the whole test suite in Red Hat UBI 9 with OpenSSL's FIPS provider forced, after first checking that FIPS is really in force.

Source: DATA_PROTECTION.md#cryptography, DATA_PROTECTION.md#on-a-fips-validated-module, guide 57

What falls outside the FIPS boundary?

Answer: The following bring their own cryptography:

- `psycopg2-binary`, which ships its own libpq and OpenSSL; build `psycopg2` from source on a FIPS platform;
- `mysql-connector-python`'s C extension;
- DuckDB's TLS for remote files;

- the Java runtime used by JDBC and H2 connections;
- the `cryptography` wheel from PyPI.

Deployer: decide which of these to use on a FIPS platform.

Source: DATA_PROTECTION.md#on-a-fips-validated-module

Which algorithms are used, and for what?

Answer:

ALGORITHM	USED FOR
Fernet	Stored passwords
SHA-256	Keys, tokens and sessions stored as hashes; PKCE
HMAC-SHA256	The audit chain; the masking `hash` action
RS256 / HS256, or configured RS*, PS*, ES*, HS*	Signed-in users' tokens
AWS SigV4	RDS IAM tokens
The OS random source (`secrets`)	Every secret the server generates

`random` is used only where nothing depends on it being unpredictable.

Source: DATA_PROTECTION.md#cryptography

---

6. Logging, audit and monitoring

Are administrative actions logged?

Answer: Yes. Every configuration change is appended to the audit log, recording:

- the actor;
- how they authenticated (`via` : their own token, break-glass key, SSO, CLI, configuration file);
- the action and target;
- a diff of what changed.

Passwords appear only as `********` or `"changed"`, and secrets and hashes never appear. Every SSO sign-in, refusal and sign-out is recorded.

Source: THREAT_MODEL.md#audit-logging-and-data-at-rest, API.md, Audit log

---

Is the audit log tamper-evident?

Answer: Yes. It is a hash chain, HMAC-signed when `QUERYAPIGATE_AUDIT_KEY` is set. Changing, removing, inserting or reordering an entry breaks the chain, which `queryapigate audit verify` detects; a check runs every 5 minutes and raises the critical `audit_chain_broken` alert. The chain alone can't show that its newest entries were deleted, or that someone holding both the store and the key rewrote it. A copy kept outside the store is what proves those.

Source: THREAT_MODEL.md#the-audit-trail

Can audit logs be sent to a SIEM?

Answer: Yes. The audit log can be sent as syslog over TLS (RFC 5425), with optional mutual TLS, or as OTLP logs. Each entry carries its `seq` and `hash`, and `audit verify --file` checks the SIEM's copy. Delivery is at least once and survives the SIEM being down. Syslog over TCP can lose a batch if the receiver dies without closing the connection; OTLP acknowledges each batch. The stream is tested on every push against rsyslog, syslog-ng and the OpenTelemetry Collector.

Source: THREAT_MODEL.md#the-audit-trail, API.md, Streaming the audit log to a SIEM

How long are audit records retained?

Answer: The store keeps the newest `QUERYAPIGATE_AUDIT_LOG_LIMIT` entries, 500 by default, and keeps more while a SIEM output is behind. `QUERYAPIGATE_AUDIT_LOG_EXPORT_FILE` appends every entry to a file that is never capped or rewritten.

Deployer: set retention in the SIEM or export file to match your obligations.

Source: API.md, Audit log, DATA_PROTECTION.md#what-a-deployer-controls

---

Is data access (query execution) logged?

Answer: Yes. Run history records each run: when it ran, which query and version, the caller, the transport, the row count, the time taken and the status. It records parameter names but never their values. Run history is a plain table and is **not** tamper-evident, unlike the audit log.

Source: DATA_PROTECTION.md#the-store, THREAT_MODEL.md#audit-logging-and-data-at-rest

---

What monitoring and alerting is available?

Answer: The server exposes Prometheus `/metrics`, and a Grafana dashboard is bundled. A built-in alerts list (`GET /api/v1/alerts`) includes, for example:

- `open_server`;
- `break_glass_used`;
- `audit_chain_broken`;
- `audit_delivery_behind`;
- `key_expiring`;

- `secret_provider_failing`;
- `agent_unusual_volume`;
- `key_unused`.

The alerts list is meant for day-to-day operators, not as a replacement for on-call alerting.

Deployer: alert on `/metrics` from Prometheus.

Source: API.md, Alerts, DEPLOYMENT.md#6-observability

Is the metrics endpoint protected?

Answer: Only when the Deployer sets `QUERYAPIGATE_METRICS_TOKEN`; until then it is open. Its labels name API keys, queries and connections. Administrators and signed-in users appear only under the labels `admin` and `jwt`. The Helm chart sets the token by default.

Deployer: set the token wherever the server can be reached from outside.

Source: THREAT_MODEL.md#non-goals-what-this-does-not-defend-against, DEPLOYMENT.md#6-observability

---

Is distributed tracing supported, and what do traces contain?

Answer: Yes. OpenTelemetry tracing is **experimental**, and it is off unless an OTLP endpoint is set. Spans carry the route, the saved query's name, the row count, the caller's name and the caller's address. They never carry parameter values, rows, statements, headers or credentials, and a test enforces this. By default a caller's `traceparent` can raise how much is sampled; `OTEL_TRACES_SAMPLER=traceidratio` prevents that.

Source: THREAT_MODEL.md#transport, ADR 0010

---

7. Availability, backup and recovery

Does the product support high availability?

Answer: Yes, with several instances on a PostgreSQL store with Redis. Everything is then shared, so instances are interchangeable behind a load balancer. This is tested in CI with three real servers. With a SQLite store, only one instance is supported; two replicas with their own SQLite volumes are two separate servers. Without Redis, rate limits and the cache are per process, and the `instances_not_shared` alert reports it. If Redis becomes unreachable, rate limits stay enforced per process, never lifted, and the `rate_limits_not_shared` alert says so.

Source: DEPLOYMENT.md#scaling-out, THREAT_MODEL.md#non-goals-what-this-does-not-defend-against

---

How is the product's data backed up and restored?

Answer: A SQLite store is backed up with `queryapigate backup`, which uses SQLite's backup API to take a consistent copy while the server runs. A PostgreSQL store is backed up with `pg_dump`, or the database's own backups. CI backs up a store, loses it, restores it, and checks the result on every run.

Deployer: schedule backups, ship them off the host, and test restores.

Source: DEPLOYMENT.md#5-persistent-data-backups-and-restores

What does a backup not contain?

Answer: A backup doesn't contain:

- `QUERYAPIGATE_SECRET_KEY`, without which encrypted passwords are unreadable;
- the admin key;
- environment variables that connections reference;
- the connected databases;
- the Redis cache, which refills on its own.

Scoped keys are in the store as hashes and keep working after a restore.

Deployer: back up those secrets separately, and back up your databases in their own right. Encrypt backups and restrict access to them: the store holds connection details, saved SQL, key hashes, run history and the audit log.

Source: DEPLOYMENT.md#what-a-backup-of-the-store-does-not-contain

---

How are upgrades performed, and can they be rolled back?

Answer: The store's schema migrates automatically on first start. A store the version can't safely run on stops startup before anything changes. Several instances can be upgraded one at a time without downtime, one minor version at a time. Going back to an older release in place is not supported: restore the backup taken before the upgrade.

Source: DEPLOYMENT.md#upgrading, DEPLOYMENT.md#rolling-upgrades-several-instances-on-one-store

---

8. Secure development and supply chain

Is security testing part of the development process?

Answer: Yes. The test suite runs in CI. It includes:

- a property-based fuzz suite (Hypothesis) for the SQL guard and parameter binder, which always runs;
- tests that fail if a Management API route has no capability, if a span carries a value, or if the code gains a non-approved algorithm;
- a full run under OpenSSL's FIPS provider.

Source: SECURITY.md#hardening-checklist-for-deployments, THREAT_MODEL.md#administration-named-administrators-and-roles, DATA_PROTECTION.md#cryptography, INSTALLATION_AND_SETUP.md

---

Are features security-reviewed before release?

Answer: Features that guard access or data come under the stability promise only after a security review. The threat model records each review's scope and findings: for example, reviews for 0.30, 0.32 and 0.34, which found and fixed issues before release.

Source: CHANGELOG, Versioning and compatibility, THREAT_MODEL.md#console-sign-in-through-sso-adr-0007, THREAT_MODEL.md#live-events-adr-0020

Are releases signed?

Answer: Yes, from 0.31. Every image (including the `-h2` variant) and the Helm chart is signed keyless with cosign, by digest, by the repository's release workflow, and the signature is recorded in Sigstore's public log. A valid signature names the identity `.../.github/workflows/docker.yml@refs/tags/v<X.Y.Z>`. Releases before 0.31 are not signed. Guide 59 shows Sigstore's policy-controller and Kyverno admitting a signed image and refusing an unsigned one.

Deployer: run only signed releases, by digest, and enforce this with an admission policy.

Source: SECURITY.md#verify-a-release, ADR 0018, guide 59

Do you provide an SBOM and build provenance?

Answer: Yes. Each image carries an SPDX SBOM, SLSA provenance naming the commit, and a GitHub artifact attestation. The chart has a GitHub attestation. PyPI packages carry PEP 740 attestations.

Source: ADR 0018, guide 59

Are build inputs pinned?

Answer: Yes. Base images are pinned by digest and every GitHub Action by commit SHA, and a test fails on an unpinned `uses:`. Dependabot proposes updates as reviewed commits. Builds are not reproducible byte for byte.

Source: ADR 0018, THREAT_MODEL.md#on-kubernetes-the-helm-chart-adr-0013

What does a release signature not guarantee?

Answer: It guarantees only that this repository's workflow built the artifact on that tag. It doesn't guarantee that the code is correct. Someone who can push a tag can still sign a release, and the public log then records it. It also doesn't vouch for the upstream base images' own supply chain.

Source: THREAT_MODEL.md#on-kubernetes-the-helm-chart-adr-0013, ADR 0018

Is configuration managed through change control?

Answer: It can be. With configuration as code, connections, grants and keys can live in one file in git. Applying a file is an owner's capability, all or nothing, and audited; a file can't hold a literal password. Anyone who can edit the file has an owner's reach at the next start.

Deployer: protect the branch and require review.

Source: THREAT_MODEL.md#configuration-as-code-adr-0014

---

9. Vulnerability management and incident response

How are security vulnerabilities reported?

Answer: Privately, not in a public issue: through GitHub's "Report a vulnerability" form on the repository, or by email to arcswdev@gmail.com.

Source: SECURITY.md#reporting-a-vulnerability

What are your response times for reported vulnerabilities?

Answer: SECURITY.md states only that a reporter can expect an acknowledgement within a few days. The project commits to no other response or remediation time.

Source: SECURITY.md#reporting-a-vulnerability

Which versions receive security fixes?

Answer: Security fixes are made against the latest released version. Older versions are not patched.

Deployer: keep up with releases. A patch release never breaks a covered surface.

Source: SECURITY.md#supported-versions, CHANGELOG, Versioning and compatibility

---

How are security fixes communicated to customers?

Answer: In the **Security** section of the changelog entry for each release that contains a fix. Recent entries give the affected version range, what could happen, and how to check whether it happened in your deployment. A security fix that can't be made compatibly may remove a feature sooner than the deprecation process allows, and says why.

Deployer: watch the changelog's Security entries.

Source: CHANGELOG, CHANGELOG, Versioning and compatibility

Can you give recent examples of security fixes?

Answer: Yes:

- **0.34.1:** fixed `allowed_tables` not being checked on MongoDB connections (affected 0.11.0 to 0.34.0).
- **0.34.2:** fixed two MongoDB issues (affected 0.9.0 to 0.34.1). A parameter value could carry query operators. Server-side JavaScript was not refused in a projection or sort.

Each entry explains how to check run history for signs of use.

Source: CHANGELOG 0.34.1, CHANGELOG 0.34.2

What tools help a customer investigate a suspected incident?

Answer: These tools are available:

- the audit log and `queryapigate audit verify`, including `--file` for the SIEM's copy;
- run history (`GET /api/v1/history`), filterable by key;
- request ids in logs;
- the `break_glass_used` alert;
- per-key `last_used_at`.

Key secrets can be revoked at once (`active: false`), or rotated with no grace period.

Deployer: incident response for your deployment is yours.

Source: THREAT_MODEL.md#the-audit-trail, API.md, Authentication and permissions, THREAT_MODEL.md#authentication-one-shared-secret-or-many-scoped-ones

---

10. Network and hardening

Does the product set HTTP security headers?

Answer: Yes. Every response sets `X-Content-Type-Options: nosniff` and `Cache-Control: no-store`; a cached saved query is `private`. API responses also carry a CSP that allows nothing and `Referrer-Policy: no-referrer`. The server version banner is removed. HSTS is opt-in (`QUERYAPIGATE_HSTS_MAX_AGE`) and sent only over HTTPS.

Source: THREAT_MODEL.md#transport, ADR 0010

How is TLS provided?

Answer: By a reverse proxy the Deployer runs (Caddy, nginx or Traefik). QueryAPIGate does not terminate TLS itself.

Deployer: terminate TLS in front of the server, and set `QUERYAPIGATE_TRUST_PROXY` to the number of proxy hops. Leave it at `0` when clients connect directly.

Source: THREAT_MODEL.md#transport, DEPLOYMENT.md#2-compose-file, SECURITY.md#hardening-checklist-for-deployments

---

Is cross-origin (CORS) access restricted?

Answer: Yes. CORS is off unless `QUERYAPIGATE_CORS_ORIGINS` is set, and cookies are not used for API credentials. CORS controls browsers only; it is not authentication.

Deployer: list only sites you control, and never use `*` without an API key.

Source: THREAT_MODEL.md#cors, API.md, Rate limiting and CORS

Does the container run as a non-root user?

Answer: Yes. The image runs as a non-root user (UID 1000) and ships a `HEALTHCHECK`.

Source: DEPLOYMENT.md#1-the-image

What pod security does the Helm chart apply?

Answer: The chart's pods run as UID 1000, with a read-only root filesystem, every capability dropped, the RuntimeDefault seccomp profile and no service account token. The server writes only to `/data` and `/tmp`. The chart refuses secret settings in plain `config`.

Source: THREAT_MODEL.md#on-kubernetes-the-helm-chart-adr-0013, DEPLOYMENT.md#kubernetes-helm

---

Is network segmentation enforced?

Answer: The Helm chart's NetworkPolicy is **off by default**. Without it, any pod in the cluster can reach the API and the MCP server. When enabled, it limits inbound connections and restricts outbound traffic to listed destinations.

Deployer: enable it, and list your databases, Redis and SIEM. Network perimeter controls (proxy, VPN, WAF) are yours.

Source: THREAT_MODEL.md#on-kubernetes-the-helm-chart-adr-0013, THREAT_MODEL.md#non-goals-what-this-does-not-defend-against

---

How are the product's own secrets protected in Kubernetes?

Answer: Secrets come from a Kubernetes Secret: `existingSecret`, or one the chart creates. Anyone who can `get` `secrets` in the namespace, or read etcd unencrypted, can read them.

Deployer: keep RBAC narrow, encrypt Secrets at rest, and prefer `existingSecret`, which keeps values out of values files and Helm's release record.

Source: THREAT_MODEL.md#on-kubernetes-the-helm-chart-adr-0013

Is there protection against SQL injection?

Answer: Yes. Bound `:name` parameters are real parameterization: values are passed to the driver separately and never become part of the SQL text. Older `{name}` text placeholders are string substitution, restricted to a narrow set of safe characters.

Deployer: prefer `:name` parameters.

Source: THREAT_MODEL.md#query-execution-what-actually-stops-a-bad-statement

Can an administrator use the product to reach files or the product's own secrets?

Answer: Several guards prevent it. A connection can't point at the product's own store. DuckDB reads only its `allowed_paths`, which are empty by default. Local export folders are off unless the Deployer lists them. A secret reference can't name the server's own settings or cloud credentials. `${file:...}` can't read the server's secret files, service account tokens or `/proc`.

Deployer: set `QUERYAPIGATE_SECRET_ENV_VARS`.

Source: THREAT_MODEL.md#query-execution-what-actually-stops-a-bad-statement, THREAT_MODEL.md#secrets-from-secrets-managers-adr-0015

---

11. Known limits and non-goals

What does the product explicitly not defend against?

Answer: QueryAPIGate does not defend against:

- network attacks: it is not a network perimeter or a web application firewall;
- a malicious administrator: an owner, admin or developer can read every row on every connection;
- a missed statement: the read-only guard doesn't replace least-privilege database credentials;
- planted instructions: an AI agent may be steered by what it reads.

The threat model lists the rest. Run history isn't tamper-evident, connection metadata is visible to every administrator, query names can be probed, and `/metrics` is open until a token is set.

Source: THREAT_MODEL.md#non-goals-what-this-does-not-defend-against

---

What are the limits of the read-only SQL guard?

Answer: It is a regex-based classifier, not a SQL parser. It is dialect-aware and fuzz-tested, but it can never be a full parser for every server mode. On DuckDB, H2 and JDBC connections, which are opened read-write, the guard is the only lock against writes. The project assumes it could have another gap like the one already found and fixed.

Source: THREAT_MODEL.md#query-execution-what-actually-stops-a-bad-statement, SECURITY.md#hardening-checklist-for-deployments

---

Are query time limits enforced on every connection type?

Answer: No. On generic JDBC connections, `QUERYAPIGATE_QUERY_TIMEOUT` is not enforced, because JDBC has no portable way to cancel a statement; a test query ran past the limit to the end.

Deployer: set a statement timeout in the database, or in the JDBC URL if your driver supports one. Setting the timeout to `0` removes the limit everywhere.

Source: guide 3, SECURITY.md#hardening-checklist-for-deployments

Which features are experimental?

Answer: H2, JDBC and MongoDB connections; tracing with OpenTelemetry; and secrets and credentials from AWS (`${aws-sm:...}`, `aws-rds-iam`). These sit outside the compatibility promise and may change in any minor release. Each is marked in its documentation, in `/openapi.json` and in Settings, and logs a warning at startup.

Source: CHANGELOG, Versioning and compatibility

Are usage quotas a security boundary?

Answer: No. Quotas meter a contract; they don't enforce a security boundary. A caller can overshoot by the calls it has in flight, and with several instances but no Redis, by up to one quota per instance in a short burst. `max_concurrent` and the rate limit are what bound a burst.

Source: THREAT_MODEL.md#rate-limiting-and-abuse-prevention

---

12. Certifications and audits

Do you hold a SOC 2 report or ISO 27001 certification?

Answer: No. SOC 2 and ISO 27001 assess an organisation running a service. QueryAPIGate is software that runs inside the Deployer's environment, under the Deployer's controls, so there is no hosted service to certify. What it provides is the evidence your own assessment needs: this pack, the threat model, the data inventory and checks you can run on your own server.

Deployer: your own SOC 2 or ISO 27001 scope covers how you run QueryAPIGate: hosts, network, secrets, backups, access reviews and log retention.

Source: guide 57

Has the product undergone an independent third-party penetration test or audit?

Answer: The project publishes no third-party penetration test or audit report. Its own security reviews are recorded in the threat model, feature by feature, with what each looked at and what it found.

Source: THREAT_MODEL.md#console-sign-in-through-sso-adr-0007 (an example review record), ADR 0017